

The Ultimate Dynamic Programming Reference Guide

Every Pattern, Every Optimization, Every Trick

Sahil Raj (After Dark)

Contents

1	Linear, Grid & Dimensional DP	7
1.1	1D Linear DP	7
1.1.1	Frog Jump (CSES 1628 / AtCoder DP-A)	7
1.1.2	Maximum Sum of Non-Adjacent Elements (House Robber)	7
1.2	2D Grid DP	8
1.2.1	Unique Paths with Obstacles (LeetCode 63)	8
1.2.2	Minimum Path Sum in Grid	8
1.2.3	Ninja's Training (3-State 2D DP)	9
1.3	3D DP: Two Simultaneous Agents	9
1.3.1	Cherry Pickup II (LeetCode 1463)	9
1.4	3D Knapsack (Two Constraints)	10
1.5	Profile DP / Broken Profile (Tiling)	11
2	Tree DP	13
2.1	Standard Tree DP (Root Fixed)	13
2.1.1	Maximum Independent Set on Tree	13
2.1.2	Tree Diameter	13
2.2	Rerooting (In-Out DP / All-Roots DP)	14
2.2.1	Sum of Distances in Tree (LeetCode 834)	14
2.3	Tree Knapsack (DP on Subtrees with Budget)	15
2.4	Centroid Decomposition DP	15
2.5	Small-to-Large Merging DP	17
3	Interval DP & Knuth's Optimization	19
3.1	Interval DP Template	19
3.1.1	Matrix Chain Multiplication	19
3.1.2	Burst Balloons (LeetCode 312)	20
3.1.3	Minimum Cost to Merge Stones (LeetCode 1000)	20
3.1.4	Optimal BST (Knuth's Classic)	20
3.2	Knuth's Optimization (Quadrangle Inequality)	21
3.3	Palindrome DP	21
3.3.1	Longest Palindromic Subsequence	21
3.3.2	Minimum Cuts for Palindrome Partitioning	22
3.4	Game Theory DP (Minimax)	22
4	Bitmask DP & Digit DP	23
4.1	Bitmask DP	23
4.1.1	Traveling Salesman Problem (TSP)	23
4.1.2	Minimum Cost to Cover All Positions	23
4.1.3	Enumerate Subsets of a Mask	24
4.2	SOS DP (Sum Over Subsets)	24
4.3	Digit DP	25
4.3.1	Count Integers in $[1, X]$ with Digit Sum $\leq K$	25

4.3.2	Count Numbers with No Two Same Adjacent Digits	26
5	Linear DP, Knapsack, Coin Change, Beggars & Subsequences	27
5.1	The Knapsack Family	27
5.1.1	0/1 Knapsack	27
5.1.2	Unbounded Knapsack	29
5.1.3	Bounded Knapsack	30
5.1.4	Grouped Knapsack	31
5.2	Coin Change	32
5.2.1	2D Coin Change (Memoised, Shows Dependency Clearly)	32
5.2.2	1D Space-Optimized Minimum Coins	32
5.2.3	Count Ways to Make Amount (Order Doesn't Matter)	32
5.2.4	Count Ways to Make Amount (Order Matters = Permutations)	33
5.2.5	CSES: Coin Combinations I & II	33
5.3	Subset Sum Variants	33
5.3.1	Boolean Subset Sum: Can We Reach Target?	33
5.3.2	Partition into Two Equal Subsets	34
5.3.3	Minimum Subset Sum Difference	34
5.3.4	Count Subsets with Given Difference	34
5.4	Rod Cutting	34
5.5	Beggars / Stars-and-Bars DP	35
5.5.1	Standard Beggars: Distribute N Items to K Beggars (≥ 0 each)	35
5.5.2	Variant: Each Beggar Gets ≥ 1 (Compositions)	35
5.5.3	Variant: Ordered Beggars ($x_1 \leq x_2 \leq \dots \leq x_K$)	36
5.5.4	Variant: Beggars with Upper Bound ($x_i \leq U$)	36
5.6	Complete Reference: All Standard 1D/2D DP Patterns	37
6	DP Optimizations: CHT, D&C, Aliens Trick	39
6.1	Convex Hull Trick (CHT)	39
6.1.1	Monotone CHT (Deque)	39
6.1.2	Li Chao Tree (Non-Monotone Queries)	40
6.1.3	Classic CHT Problem: USACO / CF Codeforces 311E	41
6.2	Divide and Conquer DP Optimization	41
6.3	Aliens Trick (Lagrangian Relaxation / λ -Optimization)	42
7	Advanced DP: Probability, Matrix Exp, Multi-State & Graph	45
7.1	Probability & Expected Value DP	45
7.1.1	Expected Dice Rolls to Reach Sum $\geq n$	45
7.1.2	Expected Value with Self-Loops (AtCoder DP "Sushi")	45
7.2	Matrix Exponentiation DP	46
7.3	Multi-State DP	47
7.3.1	Best Time to Buy and Sell Stock with Cooldown	47
7.3.2	CSES: Counting Towers (Cayley-Hamilton Reduction)	47
7.4	Graph DP	47
7.4.1	Longest Path in a DAG	47
7.4.2	DP on DAG: Count Paths / Shortest Path with State	48
7.4.3	Shortest Path DP with Extra State (Parity / Steps)	48
7.5	Contribution DP / Linearity of Expectation	48
7.5.1	Sum of Lengths of All Subarrays	48
7.5.2	Expected Number of Inversions After Random Shuffle	49
7.6	Regular Bracket Sequence (RBS) DP	49
7.6.1	Count Valid Bracket Sequences with Open Count Tracking	49
8	String DP & Subsequences	51
8.1	Longest Common Subsequence (LCS)	51

8.2	Edit Distance	51
8.3	Longest Increasing Subsequence (LIS)	52
8.3.1	$\mathcal{O}(N^2)$ DP Approach	52
8.3.2	$\mathcal{O}(N \log N)$ Binary Search Approach	52

Chapter 1

Linear, Grid & Dimensional DP

1.1 1D Linear DP

Pattern / State Definition

State: $dp[i]$ = best answer for the first i elements.

Recurrence: $dp[i] = \min / \max / \sum$ over valid predecessors $j < i$.

Key insight: The optimal answer for i depends only on a *fixed window* or *any* previous state. If it depends on any previous state, you typically need a prefix structure.

When to use this pattern

- The problem processes elements left to right and each decision depends on a bounded number of previous decisions.
- Keywords: “minimum cost”, “maximum sum”, “count ways”, “choose at most/exactly k ”.
- The transition touches $O(1)$ or $O(k)$ previous states.

1.1.1 Frog Jump (CSES 1628 / AtCoder DP-A)

Frog at position 1, wants to reach position n . From position i it can jump to $i + 1$ or $i + 2$, paying $|h_i - h_j|$ cost. Find minimum cost.

```
1 int frogJump(vector<int>& h) {
2     int n = h.size();
3     vector<int> dp(n, INT_MAX);
4     dp[0] = 0;
5     for (int i = 1; i < n; i++) {
6         dp[i] = dp[i-1] + abs(h[i] - h[i-1]);
7         if (i >= 2)
8             dp[i] = min(dp[i], dp[i-2] + abs(h[i] - h[i-2]));
9     }
10    return dp[n-1];
11 }
```

1.1.2 Maximum Sum of Non-Adjacent Elements (House Robber)

```
1 int rob(vector<int>& nums) {
2     int n = nums.size();
3     if (n == 1) return nums[0];
4     // dp[i] = max money using houses 0..i
5     // dp[i] = max(dp[i-1], dp[i-2] + nums[i])
6     int prev2 = nums[0], prev1 = max(nums[0], nums[1]);
7     for (int i = 2; i < n; i++) {
8         int cur = max(prev1, prev2 + nums[i]);
```

```

9     prev2 = prev1; prev1 = cur;
10 }
11 return prev1;
12 }

```

Complexity

$\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space (rolling variables).

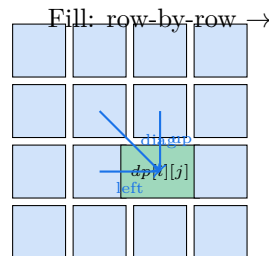
1.2 2D Grid DP

Pattern / State Definition

State: $dp[i][j]$ = best answer to reach cell (i, j) (or best answer considering the first i rows and j choices).

Fill order: Row by row, left to right (or use memoisation).

Transitions: From $(i-1, j)$, $(i, j-1)$, or diagonally from $(i-1, j-1)$.



1.2.1 Unique Paths with Obstacles (LeetCode 63)

```

1 int uniquePathsWithObstacles(vector<vector<int>>& grid) {
2     int m = grid.size(), n = grid[0].size();
3     vector<vector<long long>> dp(m, vector<long long>(n, 0));
4     // Base: first cell
5     dp[0][0] = (grid[0][0] == 0) ? 1 : 0;
6     for (int i = 1; i < m; i++) dp[i][0] = (grid[i][0] == 0) ? dp[i-1][0] : 0;
7     for (int j = 1; j < n; j++) dp[0][j] = (grid[0][j] == 0) ? dp[0][j-1] : 0;
8     for (int i = 1; i < m; i++)
9         for (int j = 1; j < n; j++)
10             dp[i][j] = (grid[i][j] == 1) ? 0 : dp[i-1][j] + dp[i][j-1];
11     return dp[m-1][n-1];
12 }

```

Optimization Note

Space Optimization (2D → 1D): Because row i only needs the values from row $i-1$ and the current row's left cell $j-1$, we can optimize the space from $\mathcal{O}(M \times N)$ to $\mathcal{O}(N)$ using a single 1D array $dp[j]$ and updating it in-place.

1.2.2 Minimum Path Sum in Grid

```

1 int minPathSum(vector<vector<int>>& grid) {
2     int m = grid.size(), n = grid[0].size();
3     // Modify in-place (space optimization)
4     for (int i = 0; i < m; i++)
5         for (int j = 0; j < n; j++) {
6             if (i == 0 && j == 0) continue;
7             int from_top = (i > 0) ? grid[i-1][j] : INT_MAX;

```

```

8         int from_left = (j > 0) ? grid[i][j-1] : INT_MAX;
9         grid[i][j] += min(from_top, from_left);
10    }
11    return grid[m-1][n-1];
12 }

```

1.2.3 Ninja's Training (3-State 2D DP)

On each day i , perform one of 3 activities (not the same as yesterday). Activity j gives $\text{points}[i][j]$ points. Maximize total.

$dp[i][j]$ = max points ending day i with activity j .

```

1 int ninjaTraining(vector<vector<int>>& points) {
2     int n = points.size();
3     // dp[j] = max points on previous day ending with activity j
4     vector<int> dp = points[0];
5     for (int i = 1; i < n; i++) {
6         vector<int> cur(3, 0);
7         for (int j = 0; j < 3; j++)           // today's activity
8             for (int k = 0; k < 3; k++)       // yesterday's activity
9                 if (k != j)
10                    cur[j] = max(cur[j], dp[k] + points[i][j]);
11        dp = cur;
12    }
13    return *max_element(dp.begin(), dp.end());
14 }

```

Optimization Note

Space Optimization (2D $\rightarrow$ 1D): Notice how the code already uses just two 1D arrays (`dp` and `cur`) instead of a full $N \times 3$ matrix. This reduces the space complexity from $\mathcal{O}(N)$ to $\mathcal{O}(1)$ (since 3 is a constant). Always look for whether $dp[i]$ only strictly depends on $dp[i-1]$.

1.3 3D DP: Two Simultaneous Agents

Pattern / State Definition

When two agents traverse the same grid simultaneously, collapse the two positions into one 3D state to avoid double-counting paths.

State: $dp[\text{step}][c_1][c_2]$ where c_1, c_2 are the column positions of agent 1 and agent 2 after `step` moves. Both agents start at the top, both move down each step (so the row is implicit: $r_1 = r_2 = \text{step}$).

1.3.1 Cherry Pickup II (LeetCode 1463)

Two robots start at top-left and top-right of an $m \times n$ grid, both move down each step. Collect max cherries without both picking the same cell.

```

1 int cherryPickup(vector<vector<int>>& grid) {
2     int m = grid.size(), n = grid[0].size();
3     // dp[c1][c2] = max cherries when robot1 at col c1, robot2 at col c2, same row
4     const int NEG_INF = -1e9;
5     vector<vector<int>> dp(n, vector<int>(n, NEG_INF));
6     dp[0][n-1] = grid[0][0] + grid[0][n-1]; // row 0 base case
7
8     for (int row = 1; row < m; row++) {
9         vector<vector<int>> ndp(n, vector<int>(n, NEG_INF));
10        for (int c1 = 0; c1 < n; c1++) {
11            for (int c2 = 0; c2 < n; c2++) {

```

```

12         if (dp[c1][c2] == NEG_INF) continue;
13         // Each robot can move in 3 directions: left, straight, right
14         for (int d1 : {-1, 0, 1}) {
15             for (int d2 : {-1, 0, 1}) {
16                 int nc1 = c1 + d1, nc2 = c2 + d2;
17                 if (nc1 < 0 || nc1 >= n || nc2 < 0 || nc2 >= n) continue;
18                 int cherries = dp[c1][c2];
19                 cherries += grid[row][nc1];
20                 if (nc1 != nc2) cherries += grid[row][nc2]; // don't double-count
21                 ndp[nc1][nc2] = max(ndp[nc1][nc2], cherries);
22             }
23         }
24     }
25 }
26 dp = ndp;
27 }
28 int ans = 0;
29 for (auto& row : dp) ans = max(ans, *max_element(row.begin(), row.end()));
30 return max(0, ans);
31 }

```

Complexity

$\mathcal{O}(m \cdot n^2)$ time, $\mathcal{O}(n^2)$ space after rolling. The key insight is that both robots always move to the next row simultaneously, making the row implicit in the state.

1.4 3D Knapsack (Two Constraints)

Pattern / State Definition

State: $dp[i][w][v]$ = can we achieve exactly weight w and volume v using the first i items?

Or: $dp[i][w][v]$ = max value using first i items within weight $\leq W$ and volume $\leq V$.

Recurrence: Same as 0/1 Knapsack but with two capacity dimensions.

```

1 // dp[w][v] = max value with weight <= w AND volume <= v
2 int knapsack3D(int W, int V, vector<int>& weight, vector<int>& volume,
3               vector<int>& value) {
4     int n = weight.size();
5     vector<vector<int>>> dp(W+1, vector<int>(V+1, 0));
6     for (int i = 0; i < n; i++) {
7         // Traverse backwards to maintain 0/1 property
8         for (int w = W; w >= weight[i]; w--)
9             for (int v = V; v >= volume[i]; v--)
10                 dp[w][v] = max(dp[w][v], dp[w - weight[i]][v - volume[i]] + value[i]);
11     }
12     return dp[W][V];
13 }

```

Complexity

$\mathcal{O}(n \cdot W \cdot V)$ time. With space optimization (2D rolling array), space is $\mathcal{O}(W \cdot V)$.

1.5 Profile DP / Broken Profile (Tiling)

Pattern / State Definition

Used to count ways to tile an $n \times m$ grid with dominoes (or other pieces). The state is a bitmask of the “profile” — which cells in the current column are already filled by a horizontal piece from the previous column.

State: $dp[col][mask]$ = number of ways to fill columns $0 \dots col$ such that after processing column col , the cells in the next column indicated by $mask$ are pre-filled.

When to use this pattern

- Tiling problems where m is small ($m \leq 12$) and n can be large.
- The number of distinct profiles is 2^m .
- Key: for each column, try all transitions from old mask to new mask by attempting to fill the column row by row using DFS/recursion.

```

1 // Count ways to tile n x m grid with 1x2 dominoes
2 // m should be the SMALLER dimension (m <= 12)
3 long long dp[2][1 << 12]; // [parity][mask]
4
5 void generate(int col, int row, int m, int prev_mask, int cur_mask,
6             int parity, long long ndp[][1 << 12]) {
7     if (row == m) {
8         ndp[parity ^ 1][cur_mask] += dp[parity][prev_mask];
9         return;
10    }
11    // Option 1: Place a vertical domino in this row (cell (row, col))
12    // The cell is already filled if prev_mask has bit 'row' set
13    if (prev_mask & (1 << row)) {
14        // Already filled by horizontal from left; pass this row
15        generate(col, row + 1, m, prev_mask, cur_mask, parity, ndp);
16    } else {
17        // Place vertical domino: fills (row, col) and (row+1, col)
18        if (row + 1 < m && !(prev_mask & (1 << (row + 1)))) {
19            generate(col, row + 2, m, prev_mask, cur_mask, parity, ndp);
20        }
21        // Place horizontal domino: fills (row, col) and (row, col+1)
22        // This sets bit in cur_mask (pre-fills next column's row)
23        generate(col, row + 1, m, prev_mask, cur_mask | (1 << row), parity, ndp);
24    }
25 }
26
27 long long tilingCount(int n, int m) {
28     if ((long long)n * m % 2 != 0) return 0; // odd cells, impossible
29     memset(dp, 0, sizeof(dp));
30     dp[0][0] = 1;
31     int parity = 0;
32     for (int col = 0; col < n; col++, parity ^= 1) {
33         long long ndp[2][1 << 12] = {};
34         for (int mask = 0; mask < (1 << m); mask++)
35             if (dp[parity][mask])
36                 generate(col, 0, m, mask, 0, parity, ndp);
37         memcpy(dp, ndp, sizeof(dp));
38     }
39     return dp[parity][0];
40 }

```

Complexity

$\mathcal{O}(n \cdot 3^m)$ time (each transition between masks is counted once per valid pair, and there are at most 3^m valid mask pairs for an m -bit column). Memory: $\mathcal{O}(2^m)$.

Chapter 2

Tree DP

2.1 Standard Tree DP (Root Fixed)

Pattern / State Definition

Root the tree. For each node u , compute $dp[u]$ from the dp values of its children. Process via a single post-order DFS (children before parent).

State: Usually $dp[u][0/1]$ or $dp[u]$ = answer for the subtree rooted at u .

2.1.1 Maximum Independent Set on Tree

$dp[u][0]$ = max independent set size in subtree of u , *not* taking u .

$dp[u][1]$ = max independent set size in subtree of u , *taking* u .

```
1 // adj[u] = list of neighbours
2 int dp[200005][2];
3 void dfs(int u, int par, vector<vector<int>>& adj) {
4     dp[u][0] = 0;
5     dp[u][1] = 1;
6     for (int v : adj[u]) {
7         if (v == par) continue;
8         dfs(v, u, adj);
9         dp[u][0] += max(dp[v][0], dp[v][1]); // don't take u: child can be taken or not
10        dp[u][1] += dp[v][0];               // take u: child must NOT be taken
11    }
12 }
13 // Answer: max(dp[root][0], dp[root][1])
```

2.1.2 Tree Diameter

The diameter is the longest path between any two nodes. For each node u , the best path *passing through* u uses the two longest depth-paths into children.

```
1 int diameter = 0;
2 int dfs_depth(int u, int par, vector<vector<int>>& adj) {
3     int best1 = 0, best2 = 0; // two longest depths among children
4     for (int v : adj[u]) {
5         if (v == par) continue;
6         int d = dfs_depth(v, u, adj) + 1;
7         if (d > best1) { best2 = best1; best1 = d; }
8         else if (d > best2) best2 = d;
9     }
10    diameter = max(diameter, best1 + best2);
11    return best1;
12 }
```

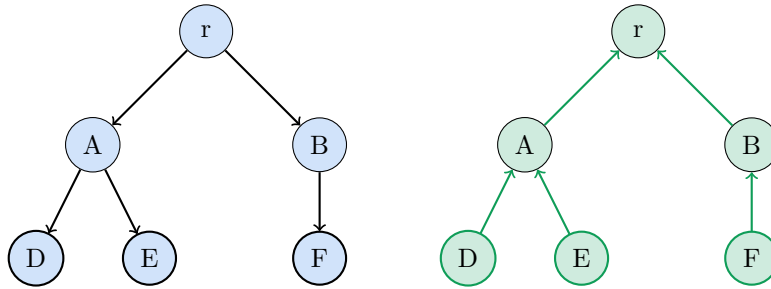
2.2 Rerooting (In-Out DP / All-Roots DP)

Pattern / State Definition

Compute, for *every* node u , the answer as if u were the root — in $\mathcal{O}(n)$ total (two DFS passes).

Pass 1 (post-order): Compute $dp_in[u]$ = answer using only the subtree of u (rooted at the original root).

Pass 2 (pre-order): Compute $dp_out[u]$ = contribution from the *rest* of the tree (everything outside u 's subtree). Combine to get the full answer for u as root.



Pass 1: dp_in (subtree)

Pass 2: dp_out (outside)

2.2.1 Sum of Distances in Tree (LeetCode 834)

For each node u , find the sum of distances to all other nodes.

```

1  int n;
2  vector<int> ans, dp_in; // dp_in[u] = sum of distances within subtree
3  vector<int> sz;        // sz[u] = subtree size
4
5  void dfs1(int u, int par, vector<vector<int>>& adj) {
6      sz[u] = 1; dp_in[u] = 0;
7      for (int v : adj[u]) {
8          if (v == par) continue;
9          dfs1(v, u, adj);
10         sz[u] += sz[v];
11         dp_in[u] += dp_in[v] + sz[v]; // each node in v's subtree contributes +1
12     }
13 }
14
15 void dfs2(int u, int par, vector<vector<int>>& adj) {
16     // ans[u] = dp_in[u] (from below) + ans[par] - sz[u] (from above, adjusted)
17     // When we reroot from par to u:
18     //   - nodes in u's subtree (sz[u] of them) get 1 closer
19     //   - nodes outside (n - sz[u]) get 1 farther
20     for (int v : adj[u]) {
21         if (v == par) continue;
22         ans[v] = ans[u] - sz[v] + (n - sz[v]);
23         dfs2(v, u, adj);
24     }
25 }
26
27 vector<int> sumOfDistancesInTree(int n_, vector<vector<int>>& edges) {

```

```

28     n = n_;
29     vector<vector<int>>> adj(n);
30     for (auto& e : edges) { adj[e[0]].push_back(e[1]); adj[e[1]].push_back(e[0]); }
31     ans.assign(n, 0); dp_in.assign(n, 0); sz.assign(n, 0);
32     dfs1(0, -1, adj);
33     ans[0] = dp_in[0];
34     dfs2(0, -1, adj);
35     return ans;
36 }

```

Complexity

$\mathcal{O}(n)$ time and space. The rerooting formula is the key: when moving the root from parent p to child v , the answer changes by $+(n - 2 \cdot \text{sz}[v])$.

2.3 Tree Knapsack (DP on Subtrees with Budget)

Pattern / State Definition

State: $dp[u][k]$ = best value achievable in the subtree of u while selecting exactly k nodes.

Transition: For each child v , merge by convolution: $dp[u][k] = \max_{j=0}^{k-1} (dp[u][k-j] + dp[v][j])$.

This is the “knapsack on a tree” and runs in $\mathcal{O}(n^2)$ using a careful size-bounded merge.

```

1  // dp[u] = vector where dp[u][k] = max value selecting k nodes in subtree of u
2  // val[u] = value of node u
3  vector<long long> tree_knapsack(int u, int par,
4      vector<vector<int>>& adj, vector<int>& val) {
5      vector<long long> dp = {0, val[u]}; // dp[0]=0, dp[1]=val[u] (just itself)
6      for (int v : adj[u]) {
7          if (v == par) continue;
8          vector<long long> child = tree_knapsack(v, u, adj, val);
9          // Merge dp (subtree so far) with child
10         int su = dp.size(), sv = child.size();
11         vector<long long> merged(su + sv - 1, -1e18);
12         for (int i = 0; i < su; i++)
13             for (int j = 0; j < sv; j++)
14                 merged[i + j] = max(merged[i + j], dp[i] + child[j]);
15         dp = merged;
16     }
17     return dp;
18 }
19 // Call: tree_knapsack(root, -1, adj, val)[k] = max value selecting k nodes

```

Complexity

Naively $\mathcal{O}(n^2k)$, but since each pair of nodes (u, v) is merged only once (when their LCA is processed), the total work is $\mathcal{O}(n^2)$ over the entire tree.

2.4 Centroid Decomposition DP

Pattern / State Definition

Count paths in the tree with a given property. Centroid decomposition splits the tree into $\mathcal{O}(\log n)$ layers. For each centroid c , count paths that pass *through* c , then recurse on the subtrees.

Key property: Every path passes through exactly one centroid in its decomposition level.

```

1  int sz[200005];
2  bool removed[200005];
3  map<int, long long> ans_map; // or array if values are bounded
4
5  int get_sz(int u, int par, vector<vector<int>>& adj) {
6      sz[u] = 1;
7      for (int v : adj[u])
8          if (v != par && !removed[v]) sz[u] += get_sz(v, u, adj);
9      return sz[u];
10 }
11
12 int get_centroid(int u, int par, int tree_sz, vector<vector<int>>& adj) {
13     for (int v : adj[u])
14         if (v != par && !removed[v] && sz[v] > tree_sz / 2)
15             return get_centroid(v, u, tree_sz, adj);
16     return u;
17 }
18
19 // Collect all depths/distances from centroid c into paths
20 void collect(int u, int par, int dist, vector<int>& depths,
21             vector<vector<int>>& adj) {
22     depths.push_back(dist);
23     for (int v : adj[u])
24         if (v != par && !removed[v]) collect(v, u, dist + 1, depths, adj);
25 }
26
27 long long count_paths_with_dist(int target, vector<int>& a, vector<int>& b) {
28     // Count pairs (x from a, y from b) with x + y == target
29     // (a = paths from previous subtrees, b = paths from current subtree)
30     map<int, int> freq;
31     for (int x : a) freq[x]++;
32     long long cnt = 0;
33     for (int y : b) if (freq.count(target - y)) cnt += freq[target - y];
34     return cnt;
35 }
36
37 void decompose(int u, int target_dist, long long& ans,
38               vector<vector<int>>& adj) {
39     int tree_sz = get_sz(u, -1, adj);
40     int c = get_centroid(u, -1, tree_sz, adj);
41     removed[c] = true;
42
43     vector<int> all_depths = {0}; // centroid itself at depth 0
44     for (int v : adj[c]) {
45         if (removed[v]) continue;
46         vector<int> depths;
47         collect(v, c, 1, depths, adj);
48         ans += count_paths_with_dist(target_dist, all_depths, depths);
49         for (int d : depths) all_depths.push_back(d);
50     }
51
52     for (int v : adj[c])
53         if (!removed[v]) decompose(v, target_dist, ans, adj);
54 }

```

Complexity

$\mathcal{O}(n \log^2 n)$ with the map, or $\mathcal{O}(n \log n)$ with sorted arrays + two pointers.

2.5 Small-to-Large Merging DP

Pattern / State Definition

Maintain a data structure (set, map, multiset) at each node. When merging children into the parent, always merge the *smaller* structure into the *larger* one. Each element is merged at most $\mathcal{O}(\log n)$ times, giving $\mathcal{O}(n \log^2 n)$ total.

When to use this pattern

- Count distinct values in each subtree, or find the k -th element.
- DP where the transition requires merging sorted structures from children.
- “Colour the tree” type problems where you need to count nodes of the majority colour.

```

1 // Count distinct values in each subtree (classic small-to-large)
2 map<int, int>* st[200005]; // st[u] = multiset of values in subtree of u
3 int ans[200005];         // ans[u] = distinct values in subtree of u
4 int val[200005];
5
6 void dfs(int u, int par, vector<vector<int>>& adj) {
7     st[u] = new map<int, int>();
8     (*st[u])[val[u]]++;
9     for (int v : adj[u]) {
10        if (v == par) continue;
11        dfs(v, u, adj);
12        // Merge smaller into larger
13        if (st[v]->size() > st[u]->size()) swap(st[u], st[v]);
14        for (auto& [key, cnt] : *st[v]) (*st[u])[key] += cnt;
15        delete st[v];
16    }
17    ans[u] = st[u]->size();
18 }
```

Complexity

$\mathcal{O}(n \log^2 n)$ due to $\mathcal{O}(\log n)$ merges per element and $\mathcal{O}(\log n)$ per map operation. Space: $\mathcal{O}(n \log n)$ total across all maps.

Common Trap / Gotcha

Don't copy, swap! The trick is to *not* create a new merged structure. Instead, swap the pointers so the larger becomes the parent's structure, then insert the smaller into it.

Chapter 3

Interval DP & Knuth's Optimization

3.1 Interval DP Template

Pattern / State Definition

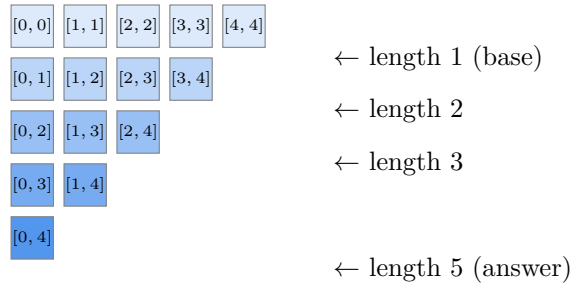
State: $dp[l][r]$ = optimal answer for the subproblem on the range $[l, r]$.

Fill order: By increasing *length* of the interval ($r - l + 1$ from 1 to n).

Transition: Split at every possible pivot k ($l \leq k < r$):

$$dp[l][r] = \min_{k=l}^{r-1} (dp[l][k] + dp[k+1][r] + \text{cost}(l, k, r))$$

Fill order: increasing length



3.1.1 Matrix Chain Multiplication

Parenthesize n matrices $M_1 M_2 \cdots M_n$ (dimensions $d_0 \times d_1, d_1 \times d_2, \dots$) to minimize total scalar multiplications.

$dp[l][r]$ = min cost to multiply $M_l \cdots M_r$.

```
1 int matrixChain(vector<int>& d) {
2     int n = d.size() - 1; // n matrices
3     vector<vector<int>> dp(n, vector<int>(n, 0));
4     // len = length of the chain being considered
5     for (int len = 2; len <= n; len++) {
6         for (int l = 0; l + len - 1 < n; l++) {
7             int r = l + len - 1;
8             dp[l][r] = INT_MAX;
9             for (int k = l; k < r; k++) // split between k and k+1
10                 dp[l][r] = min(dp[l][k],
11                                dp[l][k] + dp[k+1][r] + d[l] * d[k+1] * d[r+1]);
12         }
13     }
14     return dp[0][n-1];
15 }
```

3.1.2 Burst Balloons (LeetCode 312)

Pop balloons in $[l, r]$. If you pop balloon k last in $[l, r]$, it earns $\text{nums}[l-1] \times \text{nums}[k] \times \text{nums}[r+1]$. The “last popped” framing avoids dependency issues.

```

1 int maxCoins(vector<int>& nums) {
2     nums.insert(nums.begin(), 1);
3     nums.push_back(1);
4     int n = nums.size();
5     // dp[l][r] = max coins from popping all balloons strictly between l and r
6     vector<vector<int>> dp(n, vector<int>(n, 0));
7     for (int len = 2; len < n; len++) {
8         for (int l = 0; l + len < n; l++) {
9             int r = l + len;
10            for (int k = l + 1; k < r; k++) // k is the LAST to be popped in (l, r)
11                dp[l][r] = max(dp[l][r],
12                    dp[l][k] + dp[k][r] + nums[l] * nums[k] * nums[r]);
13        }
14    }
15    return dp[0][n-1];
16 }
```

Common Trap / Gotcha

“First popped” vs “Last popped”: The standard “first popped” framing is wrong because after popping k first, the neighbours of remaining balloons change. The “last popped” framing is stable: neighbours l and r are fixed since they are outside $[l, r]$ and not yet popped.

3.1.3 Minimum Cost to Merge Stones (LeetCode 1000)

Merge k adjacent piles into 1, costing the sum of the piles. Merge all n piles into 1. Works only if $(n-1)\%(k-1) = 0$.

```

1 int mergeStones(vector<int>& stones, int k) {
2     int n = stones.size();
3     if ((n - 1) % (k - 1) != 0) return -1;
4     vector<int> pre(n + 1, 0);
5     for (int i = 0; i < n; i++) pre[i+1] = pre[i] + stones[i];
6     // dp[l][r] = min cost to reduce stones[l..r] to as few piles as possible
7     vector<vector<int>> dp(n, vector<int>(n, 0));
8     for (int len = k; len <= n; len++) {
9         for (int l = 0; l + len - 1 < n; l++) {
10            int r = l + len - 1;
11            dp[l][r] = INT_MAX;
12            for (int mid = l; mid < r; mid += k - 1)
13                dp[l][r] = min(dp[l][r], dp[l][mid] + dp[mid+1][r]);
14            if ((len - 1) % (k - 1) == 0) // can merge into exactly 1 pile
15                dp[l][r] += pre[r+1] - pre[l];
16        }
17    }
18    return dp[0][n-1];
19 }
```

3.1.4 Optimal BST (Knuth's Classic)

Given sorted keys $k_1 < k_2 < \dots < k_n$ with search probabilities p_i and miss probabilities q_i , build a BST minimising expected search cost.

$dp[i][j]$ = min cost BST for keys $k_i \dots k_j$.

$w[i][j] = \sum_{t=i}^j p_t + \sum_{t=i-1}^j q_t$ (sum of probabilities in range).

$dp[i][j] = \min_{r=i}^j (dp[i][r-1] + dp[r+1][j]) + w[i][j]$

This satisfies the **quadrangle inequality** (see next section), so Knuth's optimization reduces this from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$.

3.2 Knuth's Optimization (Quadrangle Inequality)

Pattern / State Definition

An interval DP of the form $dp[l][r] = \min_{k=l}^{r-1} (dp[l][k] + dp[k+1][r]) + w[l][r]$ can be solved in $\mathcal{O}(n^2)$ (instead of $\mathcal{O}(n^3)$) if the cost function w satisfies the **quadrangle inequality**:

$$w[a][c] + w[b][d] \leq w[a][d] + w[b][c] \quad \text{for all } a \leq b \leq c \leq d$$

This implies the optimal split point $\text{opt}[l][r]$ is monotone:

$$\text{opt}[l][r-1] \leq \text{opt}[l][r] \leq \text{opt}[l+1][r]$$

When to use this pattern

- Optimal BST, merging files/stones with total-weight cost.
- w is the sum of elements in a range (always satisfies QI).
- Check: $w[l][r] = f(\text{prefix}[r]) - f(\text{prefix}[l-1])$ for any concave/convex f .

```

1 // Knuth's optimization template
2 // Requires: dp[l][r] = min over k of dp[l][k] + dp[k+1][r] + w[l][r]
3 // and w satisfies the quadrangle inequality
4 void knuth_dp(int n, vector<vector<int>>& dp, vector<vector<int>>& w) {
5     vector<vector<int>> opt(n+1, vector<int>(n+1, 0));
6     // Base: length 0 and length 1
7     for (int i = 0; i < n; i++) { dp[i][i] = w[i][i]; opt[i][i] = i; }
8     // Fill by increasing length
9     for (int len = 2; len <= n; len++) {
10         for (int l = 0; l + len - 1 < n; l++) {
11             int r = l + len - 1;
12             dp[l][r] = INT_MAX;
13             int lo = opt[l][r-1]; // lower bound on optimal split
14             int hi = (r > 0) ? opt[l+1][r] : r; // upper bound
15             for (int k = lo; k <= min(hi, r-1); k++) {
16                 int val = dp[l][k] + dp[k+1][r] + w[l][r];
17                 if (val < dp[l][r]) { dp[l][r] = val; opt[l][r] = k; }
18             }
19         }
20     }
21 }
```

Complexity

$\mathcal{O}(n^2)$ time (amortised: the sum of $(hi - lo)$ over all (l, r) is $\mathcal{O}(n^2)$). Space: $\mathcal{O}(n^2)$ for dp and opt .

3.3 Palindrome DP

3.3.1 Longest Palindromic Subsequence

$dp[l][r]$ = length of longest palindromic subsequence in $s[l \dots r]$.

```

1 int longestPalinSubseq(string& s) {
2     int n = s.size();
3     vector<vector<int>> dp(n, vector<int>(n, 0));
4     for (int i = 0; i < n; i++) dp[i][i] = 1;
```

```

5   for (int len = 2; len <= n; len++) {
6       for (int l = 0; l + len - 1 < n; l++) {
7           int r = l + len - 1;
8           if (s[l] == s[r]) dp[l][r] = dp[l+1][r-1] + 2;
9           else dp[l][r] = max(dp[l+1][r], dp[l][r-1]);
10      }
11  }
12  return dp[0][n-1];
13 }

```

3.3.2 Minimum Cuts for Palindrome Partitioning

$dp[l][r]$ = min cuts to partition $s[l \dots r]$ into palindromes.

```

1  int minCut(string& s) {
2      int n = s.size();
3      // isPalin[l][r] = is s[l..r] a palindrome
4      vector<vector<bool>> isPalin(n, vector<bool>(n, false));
5      for (int len = 1; len <= n; len++)
6          for (int l = 0; l + len - 1 < n; l++) {
7              int r = l + len - 1;
8              if (s[l] == s[r]) isPalin[l][r] = (len <= 2) || isPalin[l+1][r-1];
9          }
10     vector<int> dp(n, INT_MAX); // dp[i] = min cuts for s[0..i]
11     for (int r = 0; r < n; r++) {
12         if (isPalin[0][r]) { dp[r] = 0; continue; }
13         for (int l = 1; l <= r; l++)
14             if (isPalin[l][r]) dp[r] = min(dp[r], dp[l-1] + 1);
15     }
16     return dp[n-1];
17 }

```

3.4 Game Theory DP (Minimax)

Pattern / State Definition

Two players alternately pick from the ends of an array. Both play optimally.

State: $dp[l][r]$ = max score the *current player* can achieve from $a[l \dots r]$.

Transition: $dp[l][r] = \max(a[l] + \text{sum}[l+1][r] - dp[l+1][r], a[r] + \text{sum}[l][r-1] - dp[l][r-1])$

(Current player maximizes; the remaining sum minus the opponent's best score gives the current player's score from the remaining subarray.)

```

1  bool predictWinner(vector<int>& nums) {
2      int n = nums.size();
3      // dp[l][r] = max SCORE DIFFERENCE (current - opponent) from nums[l..r]
4      vector<vector<int>> dp(n, vector<int>(n, 0));
5      for (int i = 0; i < n; i++) dp[i][i] = nums[i];
6      for (int len = 2; len <= n; len++) {
7          for (int l = 0; l + len - 1 < n; l++) {
8              int r = l + len - 1;
9              dp[l][r] = max(nums[l] - dp[l+1][r], nums[r] - dp[l][r-1]);
10         }
11     }
12     return dp[0][n-1] >= 0; // first player wins if score difference >= 0
13 }

```

Complexity

All interval DP: $\mathcal{O}(n^2)$ time, $\mathcal{O}(n^2)$ space.

Chapter 4

Bitmask DP & Digit DP

4.1 Bitmask DP

Pattern / State Definition

State: $dp[mask]$ or $dp[mask][i]$ where the bitmask encodes which elements from a set of size $n \leq 20$ have been *used/visited*.

Transition: For each state, try adding one more unused element.

Key: Enumerate subsets efficiently using $mask \& (mask-1)$ or $(mask-1) \& base$.

4.1.1 Traveling Salesman Problem (TSP)

Visit every city exactly once and return to start. $n \leq 20$.

```
1 // dist[i][j] = cost to travel from city i to city j
2 // dp[mask][i] = min cost to visit exactly the cities in 'mask',
3 //               ending at city i, starting from city 0
4 int tsp(vector<vector<int>>& dist) {
5     int n = dist.size();
6     int FULL = (1 << n) - 1;
7     vector<vector<int>> dp(1 << n, vector<int>(n, 1e9));
8     dp[1][0] = 0; // Start at city 0, only city 0 visited
9     for (int mask = 1; mask < (1 << n); mask++) {
10         for (int u = 0; u < n; u++) {
11             if (!(mask & (1 << u))) continue; // u must be in mask
12             if (dp[mask][u] == 1e9) continue;
13             for (int v = 0; v < n; v++) {
14                 if (mask & (1 << v)) continue; // v already visited
15                 int nmask = mask | (1 << v);
16                 dp[nmask][v] = min(dp[nmask][v], dp[mask][u] + dist[u][v]);
17             }
18         }
19     }
20     int ans = 1e9;
21     for (int u = 1; u < n; u++) ans = min(ans, dp[FULL][u] + dist[u][0]);
22     return ans;
23 }
```

Complexity

$\mathcal{O}(2^n \cdot n^2)$ time, $\mathcal{O}(2^n \cdot n)$ space.

4.1.2 Minimum Cost to Cover All Positions

```

1 // Assign n workers to n jobs, each assignment has a cost cost[i][j].
2 // Find min cost assignment. (Hungarian is better but bitmask DP is  $O(2^n * n)$ )
3 int minCostAssignment(vector<vector<int>>& cost) {
4     int n = cost.size();
5     vector<int> dp(1 << n, INT_MAX);
6     dp[0] = 0;
7     for (int mask = 0; mask < (1 << n); mask++) {
8         if (dp[mask] == INT_MAX) continue;
9         int worker = __builtin_popcount(mask); // next worker to assign (0-indexed)
10        if (worker == n) continue;
11        for (int job = 0; job < n; job++) {
12            if (mask & (1 << job)) continue;
13            dp[mask | (1 << job)] = min(dp[mask | (1 << job)], dp[mask] + cost[worker][job]);
14        }
15    }
16    return dp[(1 << n) - 1];
17 }

```

4.1.3 Enumerate Subsets of a Mask

A classic trick: iterate over all subsets of a given mask in $O(3^n)$ total.

```

1 // Iterate over ALL subsets of 'mask' (including empty set)
2 for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
3     // process sub as a subset of mask
4 }
5 // This runs in  $O(2^{\text{popcount}(\text{mask})})$  for each mask,
6 // and  $O(3^n)$  in total across all masks.

```

4.2 SOS DP (Sum Over Subsets)

Pattern / State Definition

Compute, for every mask M : $f[M] = \sum_{S \subseteq M} a[S]$.

Naively $O(3^n)$. SOS DP achieves $O(n \cdot 2^n)$ using the inclusion principle bit by bit.

```

1 // f[mask] = sum of a[s] for all s that are subsets of mask
2 void sos_dp(vector<int>& f, int n) {
3     // f initially = a (the values)
4     for (int bit = 0; bit < n; bit++) {
5         for (int mask = 0; mask < (1 << n); mask++) {
6             if (mask & (1 << bit)) {
7                 f[mask] += f[mask ^ (1 << bit)]; // add contribution from subset without '
8                 // bit'
9             }
10        }
11    }
12    // After: f[mask] = sum of a[s] for all s subset-of mask
13 }
14 // Super-set SOS: f[mask] = sum of a[s] for all s superset-of mask
15 void sos_superset(vector<int>& f, int n) {
16     for (int bit = 0; bit < n; bit++) {
17         for (int mask = (1 << n) - 1; mask >= 0; mask--) {
18             if (!(mask & (1 << bit))) {
19                 f[mask] += f[mask | (1 << bit)];
20            }
21        }
22    }
23 }

```

Complexity $\mathcal{O}(n \cdot 2^n)$ time, $\mathcal{O}(2^n)$ space.

4.3 Digit DP

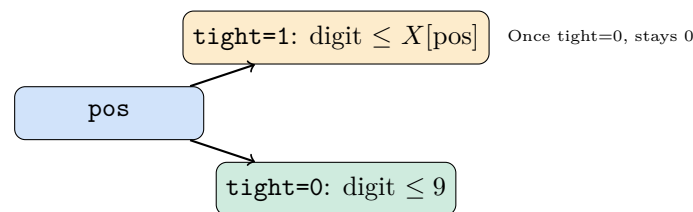
Pattern / State Definition

Count integers in $[L, R]$ satisfying some digit-based constraint. Reduce to $f(R) - f(L - 1)$ where $f(X)$ = count of valid integers in $[1, X]$.

State: $dp[pos][tight][started][extra_state]$

where:

- **pos** = current digit position (left to right)
- **tight** = are we still bounded by X 's digits? (0 or 1)
- **started** = has a non-zero digit appeared yet? (handles leading zeros)
- **extra_state** = any problem-specific state (digit sum, last digit, parity, etc.)



4.3.1 Count Integers in $[1, X]$ with Digit Sum $\leq K$

```

1 string X;
2 int K;
3 // memo[pos][sum][tight][started]
4 int memo[18][180][2][2];
5
6 int solve(int pos, int sum, bool tight, bool started) {
7     if (sum > K) return 0;
8     if (pos == (int)X.size()) return started ? 1 : 0;
9     int& ret = memo[pos][sum][tight][started];
10    if (ret != -1) return ret;
11    ret = 0;
12    int limit = tight ? (X[pos] - '0') : 9;
13    for (int d = 0; d <= limit; d++) {
14        bool new_tight = tight && (d == limit);
15        bool new_started = started || (d != 0);
16        int new_sum = (new_started) ? (sum + d) : 0; // only count if started
17        ret += solve(pos + 1, new_sum, new_tight, new_started);
18    }
19    return ret;
20 }
21
22 long long countUpTo(long long X_val) {
23     X = to_string(X_val);
24     memset(memo, -1, sizeof(memo));
25     return solve(0, 0, true, false);
26 }
27
28 // Answer for [L, R]:
29 long long answer(long long L, long long R) {
30     return countUpTo(R) - countUpTo(L - 1);
31 }
  
```

When to use this pattern

- “Count numbers in $[L, R]$ such that ... [digit property]”.
- Digit sum constraints, no two same adjacent digits, specific digit patterns.
- Numbers with at most k occurrences of digit d .
- Numbers that are NOT multiples of p (track remainder mod p in state).

4.3.2 Count Numbers with No Two Same Adjacent Digits

```

1 // State: (pos, last_digit, tight, started)
2 int memo2[18][10][2][2];
3
4 int solve2(int pos, int last, bool tight, bool started, string& X) {
5     if (pos == (int)X.size()) return started ? 1 : 0;
6     int& ret = memo2[pos][last][tight][started];
7     if (ret != -1) return ret;
8     ret = 0;
9     int limit = tight ? (X[pos] - '0') : 9;
10    for (int d = 0; d <= limit; d++) {
11        if (started && d == last) continue; // no two same adjacent
12        ret += solve2(pos + 1, d, tight && (d == limit), started || (d != 0), X);
13    }
14    return ret;
15 }
```

Common Trap / Gotcha

The leading zero trap: A number like “007” is not a 3-digit number. The **started** flag ensures we don’t count leading zeros as part of the number. Before **started** becomes true, we skip the digit-constraint checks (e.g., last digit = 0 counts as “no last digit” not digit 0).

Complexity

$\mathcal{O}(D \cdot \text{states} \cdot 10)$ where D is the number of digits. For 18-digit numbers with a digit-sum state of up to 162, this is $\mathcal{O}(18 \times 162 \times 2 \times 2 \times 10) \approx 10^5$.

Chapter 5

Linear DP, Knapsack, Coin Change, Beggars & Subsequences

This chapter covers every standard **1D and 2D linear DP** pattern that appears in competitive programming. Each pattern has a rigorous state derivation, recurrence, all meaningful variants, and traps.

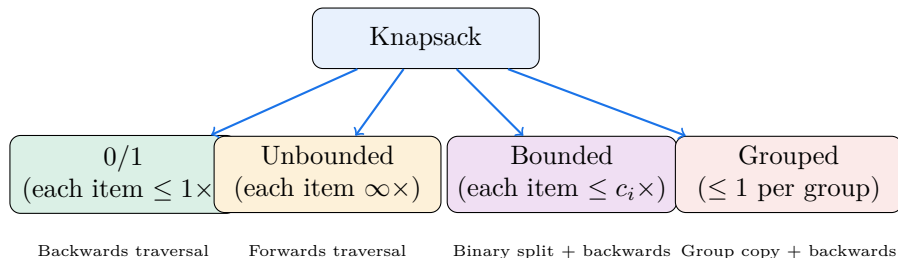
5.1 The Knapsack Family

Pattern / State Definition

All knapsack problems share one structure:

Items with weights and values. **Capacity** W . $dp[w]$ = best value achievable using capacity exactly (or at most) w .

The only difference between variants is the *inclusion rule*: 0/1 (each item once), unbounded (infinite copies), bounded (exactly c_i copies), grouped (at most 1 from each group).



5.1.1 0/1 Knapsack

Rule: Each item can be taken *at most once*.

Key: Traverse weight backwards so that each item is considered only once per capacity.

2D 0/1 Knapsack (Shows Dependency Clearly)

```
1 // 2D version (easier to understand, O(n*W) space)
2 int knapsack01_2D(int W, vector<int>& wt, vector<int>& val) {
3     int n = wt.size();
4     // dp[i][w] = max value using items 0..i-1 with weight <= w
5     vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
6     for (int i = 1; i <= n; i++) {
7         for (int w = 0; w <= W; w++) {
8             dp[i][w] = dp[i-1][w]; // don't take item i-1
9             if (w >= wt[i-1])
10                dp[i][w] = max(dp[i][w], dp[i-1][w - wt[i-1]] + val[i-1]);
11        }
```

```

12     }
13     return dp[n][W];
14 }

```

Optimization Note

Space Optimization (2D $\rightarrow$ 1D): Notice that $dp[i][w]$ only strictly requires values from the previous row $dp[i-1]$. If we traverse the weights **backwards**, we can overwrite a single 1D array $dp[w]$ in-place without accidentally reusing the same item within the same step, dropping the space complexity from $\mathcal{O}(n \cdot W)$ down to $\mathcal{O}(W)$.

1D Space-Optimized 0/1 Knapsack

```

1 // Returns max value with total weight <= W using O(W) space
2 int knapsack01(int W, vector<int>& wt, vector<int>& val) {
3     int n = wt.size();
4     vector<int> dp(W + 1, 0); // dp[w] = max value with weight <= w
5     for (int i = 0; i < n; i++)
6         for (int w = W; w >= wt[i]; w--) // BACKWARDS ensures 0/1 property
7             dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
8     return dp[W];
9 }

```

Complexity

$\mathcal{O}(n \cdot W)$ time, $\mathcal{O}(W)$ space (1D rolling), $\mathcal{O}(n \cdot W)$ space (2D).

Variant: Exact Weight Knapsack

Find the max value using *exactly* weight W (not at most).

```

1 int knapsack01_exact(int W, vector<int>& wt, vector<int>& val) {
2     vector<int> dp(W + 1, -1); // -1 = unreachable
3     dp[0] = 0;
4     for (int i = 0; i < (int)wt.size(); i++)
5         for (int w = W; w >= wt[i]; w--)
6             if (dp[w - wt[i]] != -1)
7                 dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
8     return dp[W]; // -1 if impossible
9 }

```

Variant: Count the Number of Ways

Count how many subsets achieve weight exactly W .

```

1 long long countSubsets(int W, vector<int>& wt, long long MOD = 1e9+7) {
2     vector<long long> dp(W + 1, 0);
3     dp[0] = 1; // one way to achieve weight 0 (empty set)
4     for (int w_item : wt)
5         for (int w = W; w >= w_item; w--)
6             dp[w] = (dp[w] + dp[w - w_item]) % MOD;
7     return dp[W];
8 }

```

CSES: Money Sums (All Reachable Subset Sums)

Find every distinct sum achievable by subsets of n coins. Max sum = $\sum$ coins.

```

1 void moneySums(vector<int>& coins) {
2     int total = 0;
3     for (int c : coins) total += c;
4     vector<bool> dp(total + 1, false);
5     dp[0] = true;
6     for (int c : coins)
7         for (int j = total; j >= c; j--) // backwards = 0/1
8             if (dp[j - c]) dp[j] = true;
9     vector<int> ans;
10    for (int j = 1; j <= total; j++) if (dp[j]) ans.push_back(j);
11    cout << ans.size() << "\n";
12    for (int x : ans) cout << x << " "; cout << "\n";
13 }

```

5.1.2 Unbounded Knapsack

Rule: Each item can be taken *any number of times*.

2D Unbounded Knapsack

$dp[i][w]$ = max value using items $0 \dots i - 1$ with weight $\leq w$.

```

1 int knapsackUnbounded_2D(int W, vector<int>& wt, vector<int>& val) {
2     int n = wt.size();
3     vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
4     for (int i = 1; i <= n; i++) {
5         for (int w = 0; w <= W; w++) {
6             dp[i][w] = dp[i-1][w]; // don't take item i-1
7             if (w >= wt[i-1]) // take item i-1 and stay on row i to allow reuse
8                 dp[i][w] = max(dp[i][w], dp[i][w - wt[i-1]] + val[i-1]);
9         }
10    }
11    return dp[n][W];
12 }

```

Optimization Note

Space Optimization (2D $\rightarrow$ 1D): Unlike 0/1 Knapsack, Unbounded Knapsack's $dp[i][w]$ requires $dp[i][w - wt]$ (from the *current* row i) to allow multiple selections of the same item. Because it needs updated values from the same item pass, we must traverse weights **forwards**. This lets us safely crush the 2D matrix down to 1D!

1D Space-Optimized Unbounded Knapsack

```

1 int knapsackUnbounded(int W, vector<int>& wt, vector<int>& val) {
2     vector<int> dp(W + 1, 0);
3     for (int w = 1; w <= W; w++) // FORWARDS: allows reuse
4         for (int i = 0; i < (int)wt.size(); i++)
5             if (wt[i] <= w)
6                 dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
7     return dp[W];
8 }

```

Equivalent inner-outer loop order (outer = items, inner = weights, forwards):

```

1 int knapsackUnbounded_v2(int W, vector<int>& wt, vector<int>& val) {
2     vector<int> dp(W + 1, 0);
3     for (int i = 0; i < (int)wt.size(); i++)
4         for (int w = wt[i]; w <= W; w++) // FORWARDS
5             dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
6     return dp[W];
7 }

```

Common Trap / Gotcha**Loop order matters critically for Unbounded vs 0/1:**

Variant	Inner weight loop	Reason
0/1 Knapsack	Backwards ($W \rightarrow \text{wt}[i]$)	Item used at most once
Unbounded	Forwards ($\text{wt}[i] \rightarrow W$)	Item can be reused

The *outer* loop (over items) does not change; only the inner weight direction changes.**5.1.3 Bounded Knapsack****Rule:** Item i can be taken at most c_i times.**Naive** $\mathcal{O}(n \cdot W \cdot c_{\max})$

```

1 int knapsackBounded_naive(int W, vector<int>& wt, vector<int>& val, vector<int>& cnt) {
2     vector<int> dp(W + 1, 0);
3     for (int i = 0; i < (int)wt.size(); i++)
4         for (int w = W; w >= 0; w--)
5             for (int k = 1; k <= cnt[i] && k * wt[i] <= w; k++)
6                 dp[w] = max(dp[w], dp[w - k * wt[i]] + k * val[i]);
7     return dp[W];
8 }

```

Binary Splitting $\mathcal{O}(n \cdot W \cdot \log c_{\max})$ Split c_i into powers of 2: $\{1, 2, 4, \dots, r\}$. Each group becomes a virtual 0/1 item.

```

1 int knapsackBounded_binary(int W, vector<int>& wt, vector<int>& val, vector<int>& cnt) {
2     vector<int> dp(W + 1, 0);
3     for (int i = 0; i < (int)wt.size(); i++) {
4         int rem = cnt[i];
5         // Binary split: 1, 2, 4, ..., remainder
6         for (int k = 1; rem > 0; k *= 2) {
7             int use = min(k, rem); rem -= use;
8             int bw = use * wt[i], bv = use * val[i]; // virtual 0/1 item
9             for (int w = W; w >= bw; w--) // 0/1 backwards
10                 dp[w] = max(dp[w], dp[w - bw] + bv);
11         }
12     }
13     return dp[W];
14 }

```

ComplexityNaive: $\mathcal{O}(n \cdot W \cdot c_{\max})$. Binary split: $\mathcal{O}(n \cdot W \cdot \log c_{\max})$. Monotone deque (optimal): $\mathcal{O}(n \cdot W)$ — see below.**Monotone Deque Bounded Knapsack** $\mathcal{O}(nW)$ For each item i and each residue $r = w \bmod \text{wt}[i]$, the recurrence becomes: $dp[r + k \cdot \text{wt}[i]] = \max_{j \leq \min(k, c_i)} (dp[r + (k - j) \cdot \text{wt}[i]] + j \cdot \text{val}[i])$, which is a sliding window maximum.

```

1 int knapsackBounded_deque(int W, vector<int>& wt, vector<int>& val, vector<int>& cnt) {
2     int n = wt.size();
3     vector<int> dp(W + 1, 0);
4     for (int i = 0; i < n; i++) {
5         int w_i = wt[i], v_i = val[i], c_i = cnt[i];
6         // For each residue class modulo w_i
7         for (int r = 0; r < w_i; r++) {

```

```

8      // Positions: r, r+w_i, r+2*w_i, ...
9      deque<int> dq; // stores indices k into the sequence
10     for (int k = 0; r + k * w_i <= W; k++) {
11         int w = r + k * w_i;
12         // dp[w] = max over j in [k-c_i, k] of (old_dp[r+j*w_i] + (k-j)*v_i)
13         // Equivalently: max over j of (dp[r+j*w_i] - j*v_i) + k*v_i
14         int adj = dp[w] - k * v_i; // adjusted value for sliding window
15         // Remove elements too far to the left (more than c_i steps back)
16         while (!dq.empty() && k - dq.front() > c_i) dq.pop_front();
17         // Pop elements with smaller adjusted value (monotone deque)
18         // We need to compare adjusted values stored; store (k, adj) pairs
19         // Simplified: track indices only
20         if (!dq.empty()) {
21             int j = dq.front();
22             int best = (dp[r + j * w_i] - j * v_i) + k * v_i;
23             dp[w] = max(dp[w], best);
24         }
25         // Add current k to deque (maintaining decreasing adjusted values)
26         while (!dq.empty()) {
27             int j = dq.back();
28             if ((dp[r + j * w_i] - j * v_i) <= adj)
29                 dq.pop_back();
30             else break;
31         }
32         dq.push_back(k);
33     }
34 }
35 }
36 return dp[W];
37 }

```

5.1.4 Grouped Knapsack

Rule: Items are in groups. Take at most 1 item from each group.

```

1  // groups[g] = list of (weight, value) pairs in group g
2  int groupedKnapsack(int W, vector<vector<pair<int,int>>>& groups) {
3      vector<int> dp(W + 1, 0);
4      for (auto& group : groups) {
5          // We must not update dp[w] from the same group's item into dp[w'] in the same pass
6          // So copy dp first, then update the copy from the original dp
7          vector<int> ndp = dp; // start from "take nothing from this group"
8          for (auto& [w_item, v_item] : group)
9              for (int w = W; w >= w_item; w--)
10                 ndp[w] = max(ndp[w], dp[w - w_item] + v_item); // pull from ORIGINAL dp
11         dp = ndp;
12     }
13     return dp[W];
14 }

```

Common Trap / Gotcha

Critical: Pull from the *original* dp (before any item from this group was added), not from ndp. Otherwise two items from the same group can both be selected.

5.2 Coin Change

Pattern / State Definition

Coin change is **unbounded knapsack with value = 1 and weight = coin denomination**, minimizing total weight (number of coins) instead of maximizing value.
All coin denominations can be used any number of times.

5.2.1 2D Coin Change (Memoised, Shows Dependency Clearly)

$dp[i][t]$ = min coins using coin types $0 \dots i$ to make amount t .

```

1 int coinChange_2D(vector<int>& coins, int amount) {
2     int n = coins.size();
3     // dp[i][t]: min coins using first i coin types to form amount t
4     vector<vector<int>> dp(n, vector<int>(amount + 1, 1e9));
5     for (int t = 0; t <= amount; t++)
6         dp[0][t] = (t % coins[0] == 0) ? t / coins[0] : 1e9;
7
8     for (int i = 1; i < n; i++) {
9         for (int t = 0; t <= amount; t++) {
10            dp[i][t] = dp[i-1][t]; // don't use coin i
11            if (coins[i] <= t && dp[i][t - coins[i]] != (int)1e9)
12                dp[i][t] = min(dp[i][t], dp[i][t - coins[i]] + 1); // use coin i (pulls from
13                row i, allowing reuse)
14        }
15    }
16    return dp[n-1][amount] >= 1e9 ? -1 : dp[n-1][amount];
17 }
```

Optimization Note

Space Optimization (2D $\rightarrow$ 1D): Just like Unbounded Knapsack, $dp[i][t]$ depends on $dp[i-1][t]$ (the previous coin's results) and $dp[i][t - \text{coins}[i]]$ (the current coin's results for a smaller amount). We can collapse this into a single 1D array of size $\text{amount} + 1$ by traversing amounts **forwards**.

5.2.2 1D Space-Optimized Minimum Coins

```

1 int coinChange_min(vector<int>& coins, int amount) {
2     const int INF = 1e9;
3     vector<int> dp(amount + 1, INF);
4     dp[0] = 0;
5     for (int w = 1; w <= amount; w++) // FORWARDS = unbounded
6         for (int c : coins)
7             if (c <= w && dp[w - c] != INF)
8                 dp[w] = min(dp[w], dp[w - c] + 1);
9     return dp[amount] == INF ? -1 : dp[amount];
10 }
```

5.2.3 Count Ways to Make Amount (Order Doesn't Matter)

```

1 // How many ways to make 'amount' using coins (combinations, not permutations)?
2 long long coinChange_ways(vector<int>& coins, int amount, long long MOD = 1e9+7) {
3     vector<long long> dp(amount + 1, 0);
4     dp[0] = 1;
5     // Outer = coins (ensures combinations, not permutations)
6     for (int c : coins)
7         for (int w = c; w <= amount; w++) // FORWARDS = unbounded
8             dp[w] = (dp[w] + dp[w - c]) % MOD;
9     return dp[amount];
10 }
```

5.2.4 Count Ways to Make Amount (Order Matters = Permutations)

```

1 // How many ordered sequences (permutations) of coins make 'amount'?
2 long long coinChange_perms(vector<int>& coins, int amount, long long MOD = 1e9+7) {
3     vector<long long> dp(amount + 1, 0);
4     dp[0] = 1;
5     // Outer = amounts (ensures all orderings are counted)
6     for (int w = 1; w <= amount; w++)
7         for (int c : coins)
8             if (c <= w)
9                 dp[w] = (dp[w] + dp[w - c]) % MOD;
10    return dp[amount];
11 }

```

Common Trap / Gotcha

Combinations vs Permutations: loop order is the key.

Goal	Outer loop	Inner loop
Min coins	doesn't matter	amounts, forwards
Count combinations	coins c	amounts $w \geq c$, forwards
Count permutations	amounts w	coins $c \leq w$

5.2.5 CSES: Coin Combinations I & II

I: Count ordered ways to produce sum X (permutations). Outer = amounts.

II: Count unordered ways (combinations). Outer = coins.

```

1 // CSES Coin Combinations I (permutations)
2 long long coinCombinationsI(vector<int>& coins, int X) {
3     const long long MOD = 1e9 + 7;
4     vector<long long> dp(X + 1, 0);
5     dp[0] = 1;
6     for (int w = 1; w <= X; w++)
7         for (int c : coins) if (c <= w)
8             dp[w] = (dp[w] + dp[w - c]) % MOD;
9     return dp[X];
10 }
11
12 // CSES Coin Combinations II (combinations, unordered)
13 long long coinCombinationsII(vector<int>& coins, int X) {
14     const long long MOD = 1e9 + 7;
15     vector<long long> dp(X + 1, 0);
16     dp[0] = 1;
17     for (int c : coins)
18         for (int w = c; w <= X; w++)
19             dp[w] = (dp[w] + dp[w - c]) % MOD;
20    return dp[X];
21 }

```

5.3 Subset Sum Variants

5.3.1 Boolean Subset Sum: Can We Reach Target?

```

1 bool subsetSum(vector<int>& arr, int target) {
2     vector<bool> dp(target + 1, false);
3     dp[0] = true;
4     for (int x : arr)
5         for (int j = target; j >= x; j--) // backwards = 0/1
6             dp[j] = dp[j] || dp[j - x];
7     return dp[target];

```

```
8 }
```

5.3.2 Partition into Two Equal Subsets

Problem: Can we split array into two subsets with equal sum? (LeetCode 416)

```
1 bool canPartition(vector<int>& nums) {
2     int total = 0;
3     for (int x : nums) total += x;
4     if (total % 2 != 0) return false;
5     return subsetSum(nums, total / 2);
6 }
```

5.3.3 Minimum Subset Sum Difference

```
1 int minSubsetDiff(vector<int>& arr) {
2     int total = 0;
3     for (int x : arr) total += x;
4     vector<bool> dp(total + 1, false);
5     dp[0] = true;
6     for (int x : arr)
7         for (int j = total; j >= x; j--)
8             dp[j] = dp[j] || dp[j - x];
9     // Find the largest S1 <= total/2 that is reachable
10    for (int s1 = total / 2; s1 >= 0; s1--)
11        if (dp[s1]) return total - 2 * s1;
12    return total; // only if arr is empty
13 }
```

5.3.4 Count Subsets with Given Difference

Problem: Count pairs (S_1, S_2) partitioning the array such that $S_1 - S_2 = D$.

Observation: $S_1 + S_2 = \text{total}$ and $S_1 - S_2 = D$ implies $S_1 = (\text{total} + D)/2$.

```
1 long long countSubsetsWithDiff(vector<int>& arr, int D, long long MOD = 1e9+7) {
2     int total = 0;
3     for (int x : arr) total += x;
4     if ((total + D) % 2 != 0 || total + D < 0) return 0;
5     int target = (total + D) / 2;
6     // Count subsets summing to target (0/1)
7     vector<long long> dp(target + 1, 0);
8     dp[0] = 1;
9     for (int x : arr)
10        for (int j = target; j >= x; j--)
11            dp[j] = (dp[j] + dp[j - x]) % MOD;
12    return dp[target];
13 }
```

5.4 Rod Cutting

Pattern / State Definition

A rod of length n can be cut into pieces of lengths $1, 2, \dots, n$ with known prices. Find the max revenue. This is exactly **unbounded knapsack** where weight = length, value = price, capacity = n .

```
1 int rodCutting(vector<int>& price, int n) {
2     // price[i] = revenue for a piece of length i+1 (0-indexed)
3     vector<int> dp(n + 1, 0);
4     for (int len = 1; len <= n; len++) // FORWARDS = unbounded
```

```

5     for (int cut = 1; cut <= len; cut++)
6         dp[len] = max(dp[len], price[cut - 1] + dp[len - cut]);
7     return dp[n];
8 }

```

5.5 Beggars / Stars-and-Bars DP

Pattern / State Definition

Distribute N identical items among K beggars (people). Count the ways.

This is the integer composition / partition problem solvable with a 2D DP that mirrors the combinatorial identity $\binom{N+K-1}{K-1}$ (stars and bars).

State: $dp[k][j]$ = number of ways to distribute j items to k beggars.

Transition: Beggar k gets $0, 1, 2, \dots, j$ items:

$$dp[k][j] = \sum_{t=0}^j dp[k-1][j-t] = dp[k-1][j] + dp[k][j-1]$$

(The second equality comes from prefix-sum optimization: instead of $\mathcal{O}(j)$ terms, use the running prefix from the left.)

$k \setminus j$	0	1	2	3	N
1	1	1	1	1	1
2	1	2	3	4	$N+1$
3	1	3	6	10	$\binom{N+2}{2}$

$$dp[i][j] = dp[i-1][j] \text{ (up)} + dp[i][j-1] \text{ (left)}$$

5.5.1 Standard Beggars: Distribute N Items to K Beggars (≥ 0 each)

Classic Problem

Find the number of ways to distribute N identical items among K distinct people, such that each person receives ≥ 0 items.

```

1 // Count ways to distribute N identical items to K beggars (each gets >= 0)
2 // Answer = C(N + K - 1, K - 1) = dp[K][N]
3 long long standardBeggars(int K, int N, long long MOD = 1e9 + 7) {
4     // dp[i][j] = ways to distribute j items to i beggars
5     vector<vector<long long>> dp(K + 1, vector<long long>(N + 1, 0));
6     for (int j = 0; j <= N; j++) dp[1][j] = 1; // 1 beggar: only 1 way (give all)
7     for (int i = 2; i <= K; i++) {
8         dp[i][0] = 1; // 0 items: always 1 way
9         for (int j = 1; j <= N; j++)
10             dp[i][j] = (dp[i-1][j] + dp[i][j-1]) % MOD;
11     }
12     return dp[K][N];
13 }

```

5.5.2 Variant: Each Beggar Gets ≥ 1 (Compositions)

Classic Problem

Find the number of ways to distribute N identical items among K distinct people, such that each person receives ≥ 1 item.

Give 1 item to each beggar first, reducing to distributing $N - K$ items with the standard rule.

```

1 long long beggarsAtLeastOne(int K, int N, long long MOD = 1e9 + 7) {
2     if (N < K) return 0; // impossible: can't give 1 to each
3     return standardBeggars(K, N - K, MOD); // reduce: distribute N-K with >= 0
4 }

```

5.5.3 Variant: Ordered Beggars ($x_1 \leq x_2 \leq \dots \leq x_K$)

Classic Problem

Find the number of ways to distribute N identical items among K distinct people, such that the amounts are non-decreasing: $x_1 \leq x_2 \leq \dots \leq x_K$.

Transition: $dp[k][j] = dp[k-1][j]$ (beggar k gets 0) + $dp[k][j-k]$ (every beggar gets at least 1 more).

```

1 // Count non-decreasing distributions:  $x_1 \leq x_2 \leq \dots \leq x_K$ , sum = N
2 long long orderedBeggars(int K, int N, long long MOD = 1e9 + 7) {
3     vector<vector<long long>> dp(K + 1, vector<long long>(N + 1, 0));
4     dp[0][0] = 1;
5     for (int i = 1; i <= K; i++) {
6         for (int j = 0; j <= N; j++) {
7             dp[i][j] = dp[i-1][j]; // beggar i gets 0
8             if (j >= i) // every beggar gets >= 1 more
9                 dp[i][j] = (dp[i][j] + dp[i][j-i]) % MOD;
10        }
11    }
12    return dp[K][N];
13 }

```

Optimization Note

Connection to Integer Partitions: The ordered beggars (non-decreasing distribution) with $x_1 \leq x_2 \leq \dots \leq x_K$ is equivalent to counting integer partitions of N into at most K parts. The $dp[k][j-k]$ term corresponds to subtracting 1 from every existing part (a bijection used in partition theory).

5.5.4 Variant: Beggars with Upper Bound ($x_i \leq U$)

Classic Problem

Find the number of ways to distribute N identical items among K distinct people, such that each person receives at most U items ($x_i \leq U$).

```

1 // Count ways:  $0 \leq x_i \leq U$  for all  $i$ , sum = N
2 long long beggarsWithBound(int K, int N, int U, long long MOD = 1e9 + 7) {
3     // Use inclusion-exclusion OR bounded knapsack approach
4     // Equivalently: coin change with K identical coins of value [0..U]
5     // dp[j] = ways to form sum j using k beggars
6     vector<long long> dp(N + 1, 0);
7     dp[0] = 1;
8     for (int i = 0; i < K; i++) {
9         // Prefix sum trick: dp_new[j] = sum of dp[j-U..j]
10        vector<long long> ndp(N + 1, 0);
11        vector<long long> psum(N + 2, 0);
12        for (int j = 0; j <= N; j++) psum[j+1] = (psum[j] + dp[j]) % MOD;
13        for (int j = 0; j <= N; j++) {
14            int lo = max(0, j - U);
15            ndp[j] = (psum[j+1] - psum[lo] + MOD) % MOD;
16        }
17        dp = ndp;
18    }
19    return dp[N];

```

Complexity

Standard Beggars: $\mathcal{O}(K \cdot N)$ time, $\mathcal{O}(K \cdot N)$ space (reducible to $\mathcal{O}(N)$). All variants: $\mathcal{O}(K \cdot N)$.

5.6 Complete Reference: All Standard 1D/2D DP Patterns

Problem	State	Traversal	Base case
0/1 Knapsack	$dp[w]$: max val $\leq w$	Backwards	$dp[0] = 0$
Unbounded Knapsack	$dp[w]$: max val $\leq w$	Forwards	$dp[0] = 0$
Bounded Knapsack	$dp[w]$: max val $\leq w$	Back (binary split)	$dp[0] = 0$
Grouped Knapsack	$dp[w]$: max val $\leq w$	Back, per group copy	$dp[0] = 0$
Coin Change (min)	$dp[w]$: min coins	Forwards	$dp[0] = 0$
Coin Combos (ways)	$dp[w]$: count	Outer=coins, Fwd	$dp[0] = 1$
Coin Perms (ways)	$dp[w]$: count	Outer=amounts, Fwd	$dp[0] = 1$
Subset Sum (bool)	$dp[j]$: reachable	Backwards	$dp[0] = \text{true}$
Count subsets	$dp[j]$: count	Backwards	$dp[0] = 1$
Partition equal	Subset sum to $\Sigma/2$	Backwards	—
Min subset diff	Subset sum + scan	Backwards	—
Beggars (std)	$dp[k][j]$: ways	$dp[i-1][j] + dp[i][j-1]$	$dp[1][*] = 1$
Beggars (ordered)	$dp[k][j]$: ways	$dp[k-1][j] + dp[k][j-k]$	$dp[0][0] = 1$
Rod cutting	$dp[\ell]$: max revenue	Forwards	$dp[0] = 0$
LCS	$dp[i][j]$: length	Row-by-row	$dp[0][*] = dp[*][0] = 0$
Edit distance	$dp[i][j]$: ops	Row-by-row	$dp[0][j] = j, dp[i][0] = i$
LIS (n^2)	$dp[i]$: LIS ending at i	Left-to-right	$dp[*] = 1$
LIS ($n \log n$)	tails array	Left-to-right	tails empty

Chapter 6

DP Optimizations: CHT, D&C, Aliens Trick

6.1 Convex Hull Trick (CHT)

Pattern / State Definition

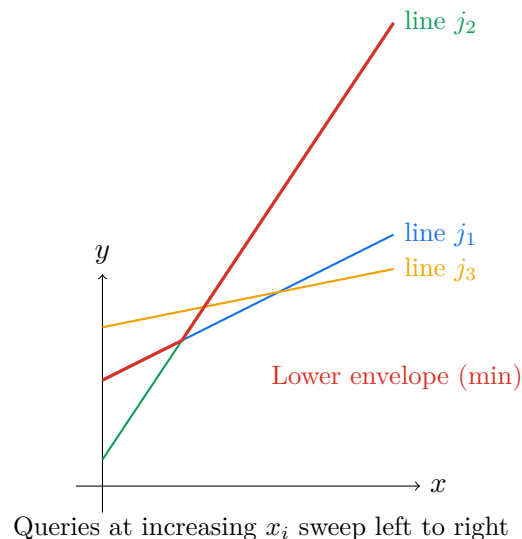
Optimize DPs of the form:

$$dp[i] = \min_{j < i} (dp[j] + m[j] \cdot x[i] + b[j])$$

where the transition is a linear function of i with a slope $m[j]$ and intercept $b[j]$ that depend only on j . Each candidate j defines a line $y = m[j] \cdot t + b[j]$. We maintain the lower envelope of these lines and query it at $x[i]$.

Monotone CHT: If both slopes are monotone and queries are monotone, use a deque in $\mathcal{O}(n)$.

Li Chao Tree: For non-monotone queries, use a segment tree over x -values in $\mathcal{O}(n \log n)$.



6.1.1 Monotone CHT (Deque)

```
1 // Lines:  $y = m \cdot x + b$ . Query: minimum  $y$  at given  $x$ .
2 // Requires: slopes are monotone (decreasing for min), queries are monotone (increasing  $x$ )
3 struct Line {
4     long long m, b;
5     long long eval(long long x) { return m * x + b; }
6 };
7
8 bool bad(Line l1, Line l2, Line l3) {
9     // l2 is never minimum if intersection(l1, l3) is left of intersection(l1, l2)
10    // Cross-multiply to avoid floating point:
```

```

11     return (__int128)(l3.b - l1.b) * (l1.m - l2.m) <= (__int128)(l2.b - l1.b) * (l1.m - l3.m
12 );
13 }
14 struct CHT {
15     deque<Line> hull;
16     void addLine(long long m, long long b) {
17         Line l = {m, b};
18         while (hull.size() >= 2 && bad(hull[hull.size()-2], hull[hull.size()-1], l))
19             hull.pop_back();
20         hull.push_back(l);
21     }
22     long long query(long long x) {
23         // If queries are monotone increasing x, pop from front
24         while (hull.size() >= 2 && hull[0].eval(x) >= hull[1].eval(x))
25             hull.pop_front();
26         return hull[0].eval(x);
27     }
28 };

```

6.1.2 Li Chao Tree (Non-Monotone Queries)

```

1  // Li Chao Segment Tree over x-values [0, MAXV]
2  const long long INF = 1e18;
3  const int MAXV = 1e6;
4
5  struct LiChao {
6      struct Line { long long m, b; long long eval(long long x) { return m*x + b; } };
7      static const Line NONE;
8      vector<Line> tree;
9      int sz;
10
11      LiChao(int n) : sz(n), tree(4*n, NONE) {}
12
13      void addLine(Line l, int node, int lo, int hi) {
14          int mid = (lo + hi) / 2;
15          bool leftBetter = l.eval(lo) < tree[node].eval(lo);
16          bool midBetter = l.eval(mid) < tree[node].eval(mid);
17          if (midBetter) swap(tree[node], l);
18          if (lo == hi) return;
19          if (leftBetter != midBetter) addLine(l, 2*node, lo, mid);
20          else addLine(l, 2*node+1, mid+1, hi);
21      }
22
23      void addLine(long long m, long long b) { addLine({m, b}, 1, 0, sz-1); }
24
25      long long query(int x, int node, int lo, int hi) {
26          long long res = tree[node].eval(x);
27          if (lo == hi) return res;
28          int mid = (lo + hi) / 2;
29          if (x <= mid) return min(res, query(x, 2*node, lo, mid));
30          else return min(res, query(x, 2*node+1, mid+1, hi));
31      }
32
33      long long query(int x) { return query(x, 1, 0, sz-1); }
34 };
35 LiChao::Line const LiChao::NONE = {0, INF};

```

Complexity

Monotone CHT: $\mathcal{O}(n)$ amortized. Li Chao Tree: $\mathcal{O}(n \log V)$ where V is the query domain. Both use $\mathcal{O}(n)$ and $\mathcal{O}(n)$ space respectively.

6.1.3 Classic CHT Problem: USACO / CF Codeforces 311E

Classic Problem

Problem Pattern: Split array into k groups. Cost of a group $[l, r]$ is $f(l, r)$ (some function). Minimize total cost.

$dp[k][r] = \min_{l \leq r} (dp[k-1][l-1] + \text{cost}(l, r))$

If $\text{cost}(l, r) = (r - l) \times c[l]$ for some c , this is a linear function of r in l , enabling CHT.

6.2 Divide and Conquer DP Optimization

Pattern / State Definition

Optimize DP of the form $dp[i][j] = \min_{k < j} (dp[i-1][k] + \text{cost}(k, j))$ when the *optimal split point* $\text{opt}[i][j]$ is monotone:

$$\text{opt}[i][j] \leq \text{opt}[i][j+1]$$

This allows a divide-and-conquer approach that computes each row in $\mathcal{O}(n \log n)$.

```

1 // dp[i] = dp for current layer (i-th group)
2 // prev[j] = dp value from previous layer
3 // cost(l, r) = cost of grouping elements l+1..r
4 long long prev_dp[5005], cur_dp[5005];
5
6 auto cost = [](int l, int r) -> long long {
7     // Define problem-specific cost here
8     return 0; // placeholder
9 };
10
11 void solve(int lo, int hi, int opt_lo, int opt_hi) {
12     if (lo > hi) return;
13     int mid = (lo + hi) / 2;
14     int best_opt = opt_lo;
15     long long best_val = LLONG_MAX;
16     for (int k = opt_lo; k <= min(mid - 1, opt_hi); k++) {
17         long long val = prev_dp[k] + cost(k, mid);
18         if (val < best_val) { best_val = val; best_opt = k; }
19     }
20     cur_dp[mid] = best_val;
21     solve(lo, mid - 1, opt_lo, best_opt);
22     solve(mid + 1, hi, best_opt, opt_hi);
23 }
24
25 // Usage: for each layer i from 1 to K:
26 //     memcpy(prev_dp, cur_dp, sizeof(cur_dp));
27 //     solve(1, n, 0, n-1);

```

Complexity

$\mathcal{O}(kn \log n)$ where k = number of layers and n = number of states per layer. Replaces $\mathcal{O}(kn^2)$.

When to use this pattern

- The DP has a “layer” structure (e.g., split into exactly k groups).
- $\text{opt}[i][j] \leq \text{opt}[i][j+1]$: monotone optimal split (harder to verify than Knuth — check numerically if unsure).
- Classic problems: APIO Sequence (partition into exactly k non-decreasing segments), CF 868F.

6.3 Aliens Trick (Lagrangian Relaxation / λ -Optimization)

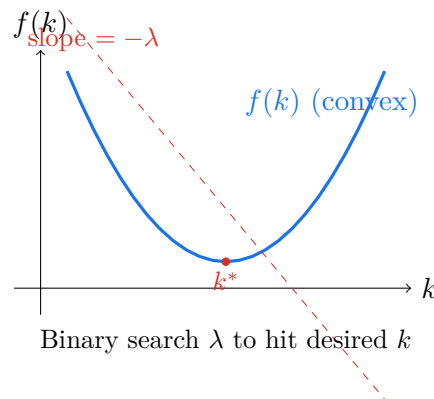
Pattern / State Definition

Problem: Compute $f(k)$ = optimal DP value when performing *exactly* k operations (where adding the k -th operation constraint makes the DP hard/impossible).

Key insight: If $f(k)$ is convex in k , binary search on a penalty λ and solve the *unconstrained* problem:

$$g(\lambda) = \min_k (f(k) + \lambda \cdot k)$$

For each λ , the unconstrained DP simultaneously finds the optimal $k(\lambda)$. Binary search on λ until $k(\lambda) =$ desired k .



```

1 // Aliens trick template
2 // solve_unconstrained(lambda) returns {min_cost, num_operations_used}
3 // for the relaxed problem where each operation costs an extra lambda.
4 // Binary search on lambda to find the lambda such that num_operations = K.
5
6 pair<long long, int> solve_unconstrained(long long lambda, /* other params */ ...) {
7     // Run the DP with an extra 'lambda' cost per operation.
8     // Track: dp[i] = {min cost, min ops to achieve that cost}
9     // Return the globally optimal (cost, num_ops).
10    return {0LL, 0}; // placeholder
11 }
12
13 long long aliens_trick(int K, /* params */ ...) {
14     long long lo = -1e12, hi = 1e12; // range for lambda
15     while (lo < hi) {
16         long long mid = lo + (hi - lo) / 2;
17         auto [cost, ops] = solve_unconstrained(mid, ...);
18         if (ops <= K) hi = mid;
19         else lo = mid + 1;
20     }
21     auto [cost, ops] = solve_unconstrained(lo, ...);
22     // Adjust: we added penalty 'lo' per operation, so true cost = cost - lo * K
23     return cost - lo * K;
24 }

```

Classic Problem

IOI 2016 Aliens / CF USACO: Cut a sequence of n elements into exactly k segments. Cost of segment $[l, r]$ is some concave/convex function. k up to 10^9 but the DP state is $\mathcal{O}(n)$. Without the Aliens trick this would be $\mathcal{O}(nk)$.

Common Trap / Gotcha

The Aliens trick only works when $f(k)$ is convex (or concave) in k . Verify this on small cases. If $f(k)$ is not convex, the binary search may return wrong answers because the tangent line may not touch every lattice point.

Complexity

$\mathcal{O}(n \log V)$ where V is the range of λ values (usually $\mathcal{O}(C_{\max})$ of individual costs). The inner DP is $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ depending on the problem.

Chapter 7

Advanced DP: Probability, Matrix Exp, Multi-State & Graph

7.1 Probability & Expected Value DP

Pattern / State Definition

Define $E[i]$ = expected value (cost/steps/score) from state i . Write the recurrence using the law of total expectation. Solve the resulting linear system when the recurrence has cycles (Markov chain). For acyclic DPs, compute directly.

7.1.1 Expected Dice Rolls to Reach Sum $\geq n$

$E[i]$ = expected rolls to get from sum i to sum $\geq n$.

$E[i] = 1 + \frac{1}{6} \sum_{d=1}^6 E[\min(i + d, n)]$, with $E[n] = 0$.

```
1 double expectedRolls(int n) {
2     vector<double> E(n + 7, 0.0);
3     // Compute from right to left
4     for (int i = n - 1; i >= 0; i--) {
5         double sum = 0;
6         for (int d = 1; d <= 6; d++) sum += E[min(i + d, n)];
7         E[i] = 1.0 + sum / 6.0;
8     }
9     return E[0];
10 }
```

7.1.2 Expected Value with Self-Loops (AtCoder DP “Sushi”)

When states can loop back to themselves (Markov chain), we get equations like: $E[i] = p \cdot E[i] + q \cdot E[i + 1] + r \cdot E[i - 1] + 1$

Rearrange: $(1 - p) \cdot E[i] = q \cdot E[i + 1] + r \cdot E[i - 1] + 1$

For complex Markov chains, set up and solve a linear system ($\mathcal{O}(n^3)$ Gaussian elimination or $\mathcal{O}(n)$ for tridiagonal systems).

```
1 // Sushi problem: n dishes, each has 1, 2, or 3 pieces. One random dish eaten each step.
2 // Expected number of steps to finish all sushi.
3 // State: (a, b, c) = number of dishes with 1, 2, 3 pieces remaining.
4 // This is a classic memoized 3D DP.
5 map<tuple<int,int,int>, double> memo;
6
7 double E(int a, int b, int c) {
```

```

8   if (a == 0 && b == 0 && c == 0) return 0;
9   auto key = make_tuple(a, b, c);
10  if (memo.count(key)) return memo[key];
11  int total = a + b + c;
12  double res = 1.0;
13  if (a > 0) res += (double)a / total * E(a - 1, b, c); // dish with 1 piece eaten
14  if (b > 0) res += (double)b / total * E(a + 1, b - 1, c); // dish with 2 -> 1 left
15  if (c > 0) res += (double)c / total * E(a, b + 1, c - 1); // dish with 3 -> 2 left
16  return memo[key] = res;
17 }

```

7.2 Matrix Exponentiation DP

Pattern / State Definition

When the DP recurrence is a *linear combination* of previous states:

$$dp[n] = c_1 \cdot dp[n-1] + c_2 \cdot dp[n-2] + \dots + c_k \cdot dp[n-k]$$

Represent it as matrix multiplication and compute A^n in $\mathcal{O}(k^3 \log n)$.

```

1  typedef vector<vector<long long>> Matrix;
2  const long long MOD = 1e9 + 7;
3
4  Matrix multiply(const Matrix& A, const Matrix& B) {
5      int n = A.size();
6      Matrix C(n, vector<long long>(n, 0));
7      for (int i = 0; i < n; i++)
8          for (int k = 0; k < n; k++) if (A[i][k])
9              for (int j = 0; j < n; j++)
10                 C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
11  return C;
12 }
13
14 Matrix matpow(Matrix A, long long p) {
15     int n = A.size();
16     Matrix result(n, vector<long long>(n, 0));
17     for (int i = 0; i < n; i++) result[i][i] = 1; // identity
18     while (p > 0) {
19         if (p & 1) result = multiply(result, A);
20         A = multiply(A, A);
21         p >>= 1;
22     }
23     return result;
24 }
25
26 // Fibonacci: F(n) = F(n-1) + F(n-2)
27 // [F(n)]   = [1 1]^(n-1) * [F(1)]   = [1 1]^(n-1) * [1]
28 // [F(n-1)] [1 0]      [F(0)]      [0]
29 long long fibonacci(long long n) {
30     if (n <= 1) return n;
31     Matrix A = {{1, 1}, {1, 0}};
32     Matrix res = matpow(A, n - 1);
33     return res[0][0]; // F(n)
34 }

```

Optimization Note

Multi-state recurrences: When you have multiple coupled linear recurrences (like Counting Towers), build the transition matrix from the system of equations and exponentiate it. The Cayley-Hamilton theorem guarantees that the characteristic polynomial of the matrix gives a single scalar recurrence for any linear combination of the states.

7.3 Multi-State DP

7.3.1 Best Time to Buy and Sell Stock with Cooldown

States: holding (own a stock), sold (just sold, in cooldown), rest (idle).

```

1 int maxProfit(vector<int>& prices) {
2     int n = prices.size();
3     // hold = max profit while holding stock
4     // sold = max profit on day just sold (next day is cooldown)
5     // rest = max profit while in cooldown or idle
6     int hold = INT_MIN, sold = 0, rest = 0;
7     for (int p : prices) {
8         int prev_hold = hold, prev_sold = sold, prev_rest = rest;
9         hold = max(prev_hold, prev_rest - p); // buy from rest
10        sold = prev_hold + p; // sell from hold
11        rest = max(prev_rest, prev_sold); // wait (from rest or after cooldown)
12    }
13    return max(sold, rest);
14 }
```

7.3.2 CSES: Counting Towers (Cayley-Hamilton Reduction)

2-state DP: $dp[i][0]$ = towers with divided top, $dp[i][1]$ = unified top. Transitions yield the linear system collapsed via the characteristic polynomial:

$$T[i] = 6 \cdot T[i-1] - 7 \cdot T[i-2]$$

```

1 void countingTowers(int n) {
2     const long long MOD = 1e9 + 7;
3     vector<long long> T(n + 1);
4     T[1] = 2; if (n >= 2) T[2] = 8;
5     for (int i = 3; i <= n; i++)
6         T[i] = (6LL * T[i-1] % MOD - 7LL * T[i-2] % MOD + 7LL * MOD) % MOD;
7     cout << T[n] << "\n";
8 }
```

7.4 Graph DP

7.4.1 Longest Path in a DAG

Topologically sort, then relax edges.

```

1 // DAG with n nodes, adjacency list. Returns longest path length.
2 int longestPathDAG(int n, vector<vector<pair<int,int>>>& adj) { // adj[u] = {v, w}
3     vector<int> indegree(n, 0);
4     for (int u = 0; u < n; u++) for (auto [v, w] : adj[u]) indegree[v]++;
5     queue<int> q;
6     for (int u = 0; u < n; u++) if (indegree[u] == 0) q.push(u);
7     vector<int> dist(n, 0);
8     while (!q.empty()) {
9         int u = q.front(); q.pop();
10        for (auto [v, w] : adj[u]) {
```

```

11         dist[v] = max(dist[v], dist[u] + w);
12         if (--indegree[v] == 0) q.push(v);
13     }
14 }
15 return *max_element(dist.begin(), dist.end());
16 }

```

7.4.2 DP on DAG: Count Paths / Shortest Path with State

```

1 // Count paths from source s to every node in a DAG
2 vector<long long> countPaths(int n, int s, vector<vector<int>>& adj) {
3     // Process in topological order (assuming 0..n-1 is already topological)
4     vector<long long> cnt(n, 0);
5     cnt[s] = 1;
6     for (int u = 0; u < n; u++)
7         for (int v : adj[u])
8             cnt[v] += cnt[u];
9     return cnt;
10 }

```

7.4.3 Shortest Path DP with Extra State (Parity / Steps)

```

1 // Shortest path from s to t using exactly K edges in a graph
2 // dp[k][u] = min cost path of exactly k edges ending at u
3 int shortestExactK(int n, int s, int K, vector<vector<pair<int,int>>& adj) {
4     const int INF = 1e9;
5     vector<vector<int>> dp(K+1, vector<int>(n, INF));
6     dp[0][s] = 0;
7     for (int k = 1; k <= K; k++)
8         for (int u = 0; u < n; u++)
9             for (auto [v, w] : adj[u])
10                 if (dp[k-1][u] < INF)
11                     dp[k][v] = min(dp[k][v], dp[k-1][u] + w);
12     return dp[K][n-1]; // or wherever target is
13 }

```

7.5 Contribution DP / Linearity of Expectation

Pattern / State Definition

Instead of computing $dp[i]$ as “best value ending at i ”, ask: *how much does element i contribute to the total answer?*

This is powerful for counting-type problems where the answer is a sum over all valid “events”.

Key identity: $E[X] = \sum_{k=1}^{\infty} P(X \geq k)$ (for non-negative integer X).

7.5.1 Sum of Lengths of All Subarrays

```

1 // Sum of lengths of all  $O(n^2)$  subarrays without iterating all of them
2 // Element  $a[i]$  contributes to  $(i+1)*(n-i)$  subarrays (choose left end in  $[0..i]$ , right in  $[i..n-1]$ )
3 long long sumOfSubarrayLengths(int n) {
4     long long total = 0;
5     for (int i = 0; i < n; i++)
6         total += (long long)(i + 1) * (n - i);
7     return total;
8 }

```

7.5.2 Expected Number of Inversions After Random Shuffle

By linearity of expectation, $E[\text{inversions}] = \binom{n}{2} \cdot \frac{1}{2}$, since each pair (i, j) is an inversion with probability $\frac{1}{2}$.

7.6 Regular Bracket Sequence (RBS) DP

Pattern / State Definition

State: $dp[n]$ = number of valid bracket sequences of length $2n$.

Catalan number recurrence: $dp[n] = \sum_{k=0}^{n-1} dp[k] \cdot dp[n-1-k]$, $dp[0] = 1$.

Closed form: $C_n = \frac{1}{n+1} \binom{2n}{n}$.

```

1 long long catalanDP(int n, long long MOD) {
2     vector<long long> dp(n + 1, 0);
3     dp[0] = 1;
4     for (int i = 1; i <= n; i++)
5         for (int k = 0; k < i; k++)
6             dp[i] = (dp[i] + dp[k] * dp[i-1-k]) % MOD;
7     return dp[n];
8 }
```

7.6.1 Count Valid Bracket Sequences with Open Count Tracking

```

1 // dp[i][j] = number of prefixes of length i with j more '(' than ')'
2 long long countRBS(int n, long long MOD) {
3     // n = total length (must be even), valid sequences have dp[n][0]
4     vector<vector<long long>> dp(n + 1, vector<long long>(n + 1, 0));
5     dp[0][0] = 1;
6     for (int i = 0; i < n; i++)
7         for (int j = 0; j <= i; j++) {
8             if (dp[i][j] == 0) continue;
9             if (j + 1 <= n) dp[i+1][j+1] = (dp[i+1][j+1] + dp[i][j]) % MOD; // add '('
10            if (j - 1 >= 0) dp[i+1][j-1] = (dp[i+1][j-1] + dp[i][j]) % MOD; // add ')'
11        }
12     return dp[n][0];
13 }
```


Chapter 8

String DP & Subsequences

8.1 Longest Common Subsequence (LCS)

Pattern / State Definition

State: $dp[i][j]$ = length of LCS of $s[0 \dots i - 1]$ and $t[0 \dots j - 1]$.

Transition: If $s[i - 1] == t[j - 1]$, $dp[i][j] = 1 + dp[i - 1][j - 1]$.
Else, $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.

```
1 int longestCommonSubsequence(string text1, string text2) {
2     int n = text1.size(), m = text2.size();
3     vector<int> prev(m + 1, 0), cur(m + 1, 0);
4     for (int i = 1; i <= n; i++) {
5         for (int j = 1; j <= m; j++) {
6             if (text1[i - 1] == text2[j - 1])
7                 cur[j] = 1 + prev[j - 1];
8             else
9                 cur[j] = max(prev[j], cur[j - 1]);
10        }
11        prev = cur;
12    }
13    return prev[m];
14 }
```

Optimization Note

Space Optimization (2D $\rightarrow$ 1D): Because row i only depends on row $i - 1$ (prev) and the current row (cur), we only need two rows instead of a full $N \times M$ matrix. Space drops from $\mathcal{O}(NM)$ to $\mathcal{O}(M)$.

8.2 Edit Distance

Pattern / State Definition

State: $dp[i][j]$ = min operations (insert, delete, replace) to convert $s[0 \dots i - 1]$ to $t[0 \dots j - 1]$.

Transition: If match: $dp[i][j] = dp[i - 1][j - 1]$.

Else: $dp[i][j] = 1 + \min(dp[i - 1][j] \text{ (del)}, dp[i][j - 1] \text{ (ins)}, dp[i - 1][j - 1] \text{ (rep)})$.

```
1 int minDistance(string word1, string word2) {
2     int n = word1.size(), m = word2.size();
3     vector<int> prev(m + 1, 0), cur(m + 1, 0);
4     for (int j = 0; j <= m; j++) prev[j] = j;
5 }
```

```

6   for (int i = 1; i <= n; i++) {
7       cur[0] = i;
8       for (int j = 1; j <= m; j++) {
9           if (word1[i - 1] == word2[j - 1])
10              cur[j] = prev[j - 1];
11           else
12              cur[j] = 1 + min({prev[j - 1], prev[j], cur[j - 1]});
13       }
14       prev = cur;
15   }
16   return prev[m];
17 }

```

Optimization Note

Space Optimization (2D $\rightarrow$ 1D): Just like LCS, Edit Distance only relies on the previous row and the current row. Using `prev` and `cur` arrays optimizes the space from $\mathcal{O}(NM)$ to $\mathcal{O}(M)$.

8.3 Longest Increasing Subsequence (LIS)

8.3.1 $\mathcal{O}(N^2)$ DP Approach

```

1   int lengthOfLIS_N2(vector<int>& nums) {
2       int n = nums.size(), ans = 1;
3       vector<int> dp(n, 1); // dp[i] = length of LIS ending at index i
4       for (int i = 1; i < n; i++) {
5           for (int j = 0; j < i; j++) {
6               if (nums[j] < nums[i])
7                   dp[i] = max(dp[i], dp[j] + 1);
8           }
9           ans = max(ans, dp[i]);
10      }
11      return ans;
12  }

```

8.3.2 $\mathcal{O}(N \log N)$ Binary Search Approach

Pattern / State Definition

Instead of $dp[i]$, maintain $tails[k] =$ the smallest ending element of all increasing subsequences of length $k + 1$. The `tails` array will naturally be strictly increasing, allowing binary search.

```

1   int lengthOfLIS_NlogN(vector<int>& nums) {
2       vector<int> tails;
3       for (int x : nums) {
4           auto it = lower_bound(tails.begin(), tails.end(), x);
5           if (it == tails.end())
6               tails.push_back(x); // Append to extend the LIS length
7           else
8               *it = x; // Replace to maintain a smaller ending value
9       }
10      return tails.size();
11  }

```